

TECHNICAL STANDARD OPERATING PROCEDURE

LAB-4: PostgreSQL 18.6 Point-in-Time Recovery (PITR) on AWS EC2 Linux

Base Backup + WAL Archiving + Named Restore Point + Recovery to DB2 (AWS EC2 Linux PostgreSQL Instance)

Item	Value
Source Server	DB1 - 172.30.2.177
Recovery Server	DB2 - 172.30.2.128
Operating System	Amazon Linux 2023
PostgreSQL	18.6
Region	AWS us-east-2 (Ohio)
PGDATA	/var/lib/pgsql/data
Lab Result	PASS - tested successfully
Tested By	Praveen Madupu
Lab Date	02 September 2026 - Wednesday

DB1 & DB2 Server / IP Details

1. Server and IP Details

#	Server	EC2 Name	LAB-4 Role	Private IP	Public IP	PostgreSQL
1	DB1	pgsqldb1	Source / Original DB	172.30.2.177	3.15.171.222	18.6
2	DB2	pgsqldb2	PITR Recovery	172.30.2.128	18.218.173.106	18.6

2. Public DNS Details

Server	Public DNS
DB1	ec2-3-15-171-222.us-east-2.compute.amazonaws.com
DB2	ec2-18-218-173-106.us-east-2.compute.amazonaws.com

3. Additional AWS and PostgreSQL Details

Parameter	DB1	DB2
Linux Hostname	db1	db2
EC2 Instance ID	i-05f19cb92d48920ba	i-00d2d4cca2f8392ee
Instance Type	t3.micro	t3.micro
Operating System	Amazon Linux 2023	Amazon Linux 2023
AWS Region	us-east-2 (Ohio)	us-east-2 (Ohio)
VPC	vpc-06ecfe6c3bb295eff	vpc-06ecfe6c3bb295eff
Subnet	subnet-08493b7b34a2e8476	subnet-08493b7b34a2e8476
Security Group	launch-wizard-1	launch-wizard-1
Security Group ID	sg-074f004964aed8446	sg-074f004964aed8446
PostgreSQL Port	5432	5432
PGDATA	/var/lib/pgsql/data	/var/lib/pgsql/data
PostgreSQL Service	postgresql.service	postgresql.service
LAB-4 Role	Source / Post-DELETE DB	PITR Recovered DB
Final Data State	0 rows after DELETE	3 valid rows recovered
PITR Timeline	Timeline 3	Promoted to Timeline 4

4. LAB-4 Recovery Flow

Stage	Details
Source	DB1 / pgsqldb1 / 172.30.2.177 / PostgreSQL 18.6
Backup & WAL	Physical base backup + archived WAL
Recovery Target	lab4_before_accidental_delete
PITR Recovery	Replay WAL only up to named restore point
Recovered Server	DB2 / pgsqldb2 / 172.30.2.128 / PostgreSQL 18.6
Timeline	Timeline 3 -> Timeline 4

Index

#	Section	Page
1	Purpose and Final Outcome	3
2	Lab Architecture	3
3	Environment and Prerequisites	4
4	PITR Concepts Used in This Lab	4
5	Phase 1 - Validate DB1 Source	5
6	Phase 2 - Configure WAL Archiving	5
7	Phase 3 - Take Physical Base Backup	6
8	Phase 4 - Create PITR Test Data and Restore Point	7
9	Phase 5 - Simulate Accidental DELETE and Archive WAL	8
10	Phase 6 - Package and Transfer Backup Files	9
11	Phase 7 - Preserve DB2 and Restore Base Backup/WAL	10
12	Phase 8 - Configure PostgreSQL 18 PITR on DB2	12
13	Phase 9 - Start Recovery and Validate	13
14	Final DB1 vs DB2 Validation	15
15	Acceptance Checklist	16
16	Post-Lab Cleanup and Production Notes	17
17	Official PostgreSQL References	18

1. Purpose and Final Outcome

This lab demonstrates PostgreSQL 18.6 Point-in-Time Recovery using a physical base backup and continuously archived WAL. DB1 remains the source database containing the post-incident state, while DB2 is rebuilt from the base backup and WAL is replayed only up to the named restore point created immediately before the accidental DELETE.

- Recovery target: lab4_before_accidental_delete
- Restore point LSN: 0/C027D38
- Source timeline before PITR: 3
- Recovered DB2 timeline after promotion: 4
- Final DB2 state: PostgreSQL active, pg_is_in_recovery() = false, transaction_read_only = off
- Final result: the three valid rows were recovered and the later accidental DELETE was excluded.

2. Lab Architecture

```

DB1 - SOURCE / POST-INCIDENT DATABASE
172.30.2.177
PostgreSQL 18.6
    |
    | pg_basebackup + archived WAL
    v
/var/lib/pgsql/pitr_backups/lab4_basebackup
/var/lib/pgsql/wal_archive
    |
    | transfer through administrator workstation
    v
DB2 - PITR RECOVERY SERVER
172.30.2.128
PostgreSQL 18.6
    |
    | restore base backup
    | replay archived WAL
    | stop at named restore point
    v
lab4_before_accidental_delete
    |
    | recovery_target_action = 'promote'
    v
NEW TIMELINE 4 / WRITABLE RECOVERED DATABASE

```

3. Environment and Prerequisites

Component	DB1	DB2
Role during LAB-4	Source / damaged current state	PITR recovery server
Hostname	db1	db2
Private IP	172.30.2.177	172.30.2.128
OS	Amazon Linux 2023	Amazon Linux 2023
PostgreSQL	18.6	18.6
PGDATA	/var/lib/pgsql/data	/var/lib/pgsql/data
Service	postgresql.service	postgresql.service

- The LAB-3 DB2 standby data directory must be preserved before repurposing DB2 for PITR.
- The source and recovery host must use compatible PostgreSQL 18 server binaries and the physical backup must remain intact.
- WAL required from the base-backup start point through the recovery target must be available.
- PostgreSQL 18 uses recovery.signal and normal server configuration parameters; recovery.conf is not used.

4. PITR Concepts Used in This Lab

Component	Lab Implementation
Base backup	pg_basebackup in plain format with streamed WAL
WAL archive	/var/lib/pgsql/wal_archive
Archive command	test for target file, then copy WAL to archive
Recovery target	Named restore point: lab4_before_accidental_delete
Restore command	Copy requested WAL from archive into PostgreSQL recovery path
Recovery trigger	recovery.signal
Target action	promote

5. Phase 1 - Validate DB1 Source

Run on DB1:

```
hostname
hostname -I
sudo systemctl is-active postgresql

sudo -u postgres psql -c "
SELECT version();
"

sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_standby,
    current_setting('transaction_read_only') AS read_only,
    pg_current_wal_lsn() AS current_wal_lsn;
"
```

Validated result: DB1 was active, PostgreSQL 18.6, not in recovery, writable, and initially at WAL LSN 0/8000778.

6. Phase 2 - Configure WAL Archiving

```
sudo mkdir -p /var/lib/pgsql/wal_archive
sudo chown postgres:postgres /var/lib/pgsql/wal_archive
sudo chmod 700 /var/lib/pgsql/wal_archive

sudo cp /var/lib/pgsql/data/postgresql.conf /var/lib/pgsql/config_backups/postgresql.conf.before_lab4_pitr

sudo -u postgres psql -c "ALTER SYSTEM SET wal_level = 'replica';"
sudo -u postgres psql -c "ALTER SYSTEM SET archive_mode = 'on';"
sudo -u postgres psql -c "
ALTER SYSTEM SET archive_command =
'test ! -f /var/lib/pgsql/wal_archive/%f && cp %p /var/lib/pgsql/wal_archive/%f';
"

sudo systemctl restart postgresql

sudo -u postgres psql -c "
SELECT name, setting
FROM pg_settings
WHERE name IN ('wal_level', 'archive_mode', 'archive_command')
ORDER BY name;
"

sudo -u postgres psql -c "SELECT pg_switch_wal();"
sleep 5

sudo ls -lh /var/lib/pgsql/wal_archive

sudo -u postgres psql -c "
SELECT
    archived_count,
    last_archived_wal,
    last_archived_time,
    failed_count,
    last_failed_wal,
    last_failed_time
FROM pg_stat_archiver;
"
```

Validated: archive_mode = on, wal_level = replica, archive command active, and failed_count = 0.

7. Phase 3 - Take Physical Base Backup

```

sudo mkdir -p /var/lib/pgsql/pitr_backups
sudo chown postgres:postgres /var/lib/pgsql/pitr_backups
sudo chmod 700 /var/lib/pgsql/pitr_backups

sudo -u postgres pg_basebackup -D /var/lib/pgsql/pitr_backups/lab4_basebackup -Fp -X stream -P

sudo cat /var/lib/pgsql/pitr_backups/lab4_basebackup/PG_VERSION
sudo cat /var/lib/pgsql/pitr_backups/lab4_basebackup/backup_label

```

Successful base backup output:

32419/32419 kB (100%), 1/1 tablespace

Backup Label Field	Validated Value
START WAL LOCATION	0/B000028 (00000003000000000000000B)
CHECKPOINT LOCATION	0/B000080
BACKUP METHOD	streamed
BACKUP FROM	primary
START TIME	2026-09-02 21:26:06 UTC
START TIMELINE	3

8. Phase 4 - Create PITR Test Data and Restore Point

Create the lab table and three valid rows on DB1:

```
sudo -u postgres psql -d labdb -c "
CREATE TABLE IF NOT EXISTS pitr_lab4
(
    id            integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    message       text NOT NULL,
    created_at    timestamptz DEFAULT clock_timestamp()
);
"

sudo -u postgres psql -d labdb -c "
INSERT INTO pitr_lab4(message)
VALUES
('LAB-4: Valid Record 1'),
('LAB-4: Valid Record 2'),
('LAB-4: Valid Record 3');
"

sudo -u postgres psql -d labdb -c "
SELECT *
FROM pitr_lab4
ORDER BY id;
"
```

Create the deterministic recovery target:

```
sudo -u postgres psql -c "
SELECT pg_create_restore_point(
    'lab4_before_accidental_delete'
);
"

sudo -u postgres psql -c "
SELECT
    clock_timestamp() AS recovery_reference_time,
    pg_current_wal_lsn() AS recovery_reference_lsn;
"
```

Item	Observed Value
Named restore point	lab4_before_accidental_delete
Restore point LSN	0/C027D38
Reference time captured	2026-09-02 21:27:05.07223+00

9. Phase 5 - Simulate Accidental DELETE and Archive WAL

Capture the pre-incident state, perform one accidental DELETE, and force the relevant WAL to archive.

```

sudo -u postgres psql -d labdb -c "
SELECT
    clock_timestamp() AS before_delete_time,
    pg_current_wal_lsn() AS before_delete_lsn;
"

sudo -u postgres psql -d labdb -c "
DELETE FROM pitr_lab4;
"

sudo -u postgres psql -c "
SELECT
    clock_timestamp() AS after_delete_time,
    pg_current_wal_lsn() AS after_delete_lsn;
"

sudo -u postgres psql -c "SELECT pg_switch_wal();"
sudo -u postgres psql -c "SELECT pg_switch_wal();"

sudo -u postgres psql -c "
SELECT
    archived_count,
    last_archived_wal,
    last_archived_time,
    failed_count
FROM pg_stat_archiver;
"

sudo ls -lh /var/lib/pgsql/wal_archive

```

Checkpoint	Observed
Before DELETE LSN	0/C027D70
DELETE result	DELETE 3
After DELETE LSN	0/C027E78
Last archived WAL	000000030000000000000000C
Archive failed_count	0

The archive contained WAL segments 0008, 0009, 000A, 000B, the base-backup .backup file, and 000C. Segment 000C contains the recovery target and later DELETE history; the named recovery target prevents replay from continuing through the bad transaction.

10. Phase 6 - Package and Transfer Backup Files

Package on DB1:

```
sudo tar -czf /tmp/lab4_basebackup.tar.gz -C /var/lib/pgsql/pitr_backups lab4_basebackup

sudo chown ec2-user:ec2-user /tmp/lab4_basebackup.tar.gz

sudo tar -czf /tmp/lab4_wal_archive.tar.gz -C /var/lib/pgsql wal_archive

sudo chown ec2-user:ec2-user /tmp/lab4_wal_archive.tar.gz

ls -lh /tmp/lab4_*.tar.gz
```

Validated package sizes were approximately 4.2 MB for the compressed base backup and 124 KB for the compressed WAL archive.

10.1 Transfer through the Windows administrator workstation

Run from Windows PowerShell in the directory containing pgsqlabssep2026.pem:

```
mkdir C:\PGSQL_LAB4_PITR

scp -i "C:\Users\DELL\Downloads\pgsqlabssep2026.pem" \
ec2-user@3.15.171.222:/tmp/lab4_basebackup.tar.gz \
C:\PGSQL_LAB4_PITR\

scp -i "C:\Users\DELL\Downloads\pgsqlabssep2026.pem" \
ec2-user@3.15.171.222:/tmp/lab4_wal_archive.tar.gz \
C:\PGSQL_LAB4_PITR\

scp -i "C:\Users\DELL\Downloads\pgsqlabssep2026.pem" \
C:\PGSQL_LAB4_PITR\lab4_basebackup.tar.gz \
ec2-user@18.218.173.106:/tmp/

scp -i "C:\Users\DELL\Downloads\pgsqlabssep2026.pem" \
C:\PGSQL_LAB4_PITR\lab4_wal_archive.tar.gz \
ec2-user@18.218.173.106:/tmp/
```

11. Phase 7 - Preserve DB2 and Restore Base Backup/WAL

Run on DB2. Stop PostgreSQL and preserve the previous LAB-3 standby before replacing PGDATA.

```
sudo systemctl stop postgresql
sudo systemctl is-active postgresql

sudo mv /var/lib/pgsql/data /var/lib/pgsql/data_before_lab4_pitr_20260902

sudo mkdir -p /var/lib/pgsql/data
sudo chown postgres:postgres /var/lib/pgsql/data
sudo chmod 700 /var/lib/pgsql/data
```

11.1 Restore the physical base backup

```
sudo tar -xzf /tmp/lab4_basebackup.tar.gz -C /var/lib/pgsql/data --strip-components=1

sudo chown -R postgres:postgres /var/lib/pgsql/data
sudo chmod 700 /var/lib/pgsql/data

sudo cat /var/lib/pgsql/data/PG_VERSION
sudo cat /var/lib/pgsql/data/backup_label
```

11.2 Restore the archived WAL repository

```
sudo tar -xzf /tmp/lab4_wal_archive.tar.gz -C /var/lib/pgsql

sudo chown -R postgres:postgres /var/lib/pgsql/wal_archive
sudo chmod 700 /var/lib/pgsql/wal_archive

sudo find /var/lib/pgsql/wal_archive -maxdepth 1 -type f -printf '%f
' | sort
```

Validated WAL archive contents:

```
000000030000000000000008
000000030000000000000009
00000003000000000000000A
00000003000000000000000B
00000003000000000000000B.00000028.backup
00000003000000000000000C
```

12. Phase 8 - Configure PostgreSQL 18 PITR on DB2

The restored base backup inherited replication settings from its source history. For this PITR recovery, remove the old primary connection settings and configure archive recovery.

```
sudo cp /var/lib/pgsql/data/postgresql.auto.conf /var/lib/pgsql/data/postgresql.auto.conf.before_lab4_pitr

sudo sed -i '/^[:space:]]*primary_conninfo[:space:]]*=/d' /var/lib/pgsql/data/postgresql.auto.conf

sudo sed -i '/^[:space:]]*primary_slot_name[:space:]]*=/d' /var/lib/pgsql/data/postgresql.auto.conf

sudo sed -i '/^[:space:]]*restore_command[:space:]]*=/d' /var/lib/pgsql/data/postgresql.auto.conf

sudo sed -i '/^[:space:]]*recovery_target/d' /var/lib/pgsql/data/postgresql.auto.conf

sudo tee -a /var/lib/pgsql/data/postgresql.auto.conf >/dev/null <<'EOF'
restore_command = 'cp /var/lib/pgsql/wal_archive/%f %p'
recovery_target_name = 'lab4_before_accidental_delete'
recovery_target_action = 'promote'
EOF

sudo chown postgres:postgres /var/lib/pgsql/data/postgresql.auto.conf

sudo chmod 600 /var/lib/pgsql/data/postgresql.auto.conf

sudo rm -f /var/lib/pgsql/data/standby.signal

sudo touch /var/lib/pgsql/data/recovery.signal
sudo chown postgres:postgres /var/lib/pgsql/data/recovery.signal
sudo chmod 600 /var/lib/pgsql/data/recovery.signal
```

Final effective PITR entries:

```
restore_command = 'cp /var/lib/pgsql/wal_archive/%f %p'
recovery_target_name = 'lab4_before_accidental_delete'
recovery_target_action = 'promote'
```

No active primary_conninfo, primary_slot_name, or standby.signal should remain.

12.1 Pre-start validation

```
sudo grep -nE "primary_conninfo|primary_slot_name|restore_command|recovery_target"
/var/lib/pgsql/data/postgresql.auto.conf

sudo ls -l /var/lib/pgsql/data/*signal
sudo cat /var/lib/pgsql/data/PG_VERSION
sudo systemctl is-active postgresql

sudo find /var/lib/pgsql/wal_archive -maxdepth 1 -type f | wc -l
```

13. Phase 9 - Start Recovery and Validate

```
sudo systemctl start postgresql
sudo systemctl status postgresql --no-pager -l

sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_in_recovery,
    current_setting('transaction_read_only') AS read_only;
"

sudo -u postgres psql -d labdb -c "
SELECT *
FROM pitr_lab4
ORDER BY id;
"
```

Successful recovery evidence from the PostgreSQL log:

```
LOG: starting backup recovery with redo LSN 0/B000028, checkpoint LSN 0/B000080, on timeline ID 3
LOG: restored log file "00000003000000000000000B" from archive
LOG: starting point-in-time recovery to "lab4_before_accidental_delete"
LOG: restored log file "00000003000000000000000C" from archive
LOG: consistent recovery state reached at 0/B000158
LOG: recovery stopping at restore point "lab4_before_accidental_delete"
LOG: selected new timeline ID: 4
LOG: archive recovery complete
LOG: database system is ready to accept connections
```

Use the actual configured log file to capture recovery evidence:

```
sudo sh -c '
grep -Ei "restore|recovery|redo|consistent|restore point|timeline|promot|ready to accept"
/var/lib/pgsql/data/log/postgresql-Wed.log | tail -100
'
```

13.1 Confirm recovery mode, writability, and timeline

```
hostname
sudo systemctl is-active postgresql

sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery(),
    current_setting('transaction_read_only');
"

sudo -u postgres psql -c "
SELECT
    timeline_id,
    redo_lsn,
    checkpoint_lsn
FROM pg_control_checkpoint();
"
```

Validation	Final Observed Value
Hostname	db2
Service	active
pg_is_in_recovery()	false
transaction_read_only	off
timeline_id	4
Final observed redo_lsn	0/C032A30
Final observed checkpoint_lsn	0/C032A88

14. Final DB1 vs DB2 Validation

14.1 DB1 - post-incident source state

```
sudo -u postgres psql -d labdb -c "
SELECT
    id,
    message,
    created_at
FROM pitr_lab4
ORDER BY id;
"
```

Observed on DB1: 0 rows. This is the expected post-DELETE state.

14.2 DB2 - recovered state

```
sudo -u postgres psql -d labdb -c "
SELECT
    id,
    message,
    created_at
FROM pitr_lab4
ORDER BY id;
"
```

Expected/validated PITR state before any post-recovery insert:

```
1 | LAB-4: Valid Record 1
2 | LAB-4: Valid Record 2
3 | LAB-4: Valid Record 3
```

14.3 Prove DB2 is writable after promotion

```
sudo -u postgres psql -d labdb -c "
INSERT INTO pitr_lab4(message)
VALUES ('LAB-4: Record Created After Successful PITR');
"

sudo -u postgres psql -d labdb -c "
SELECT
    id,
    message,
    created_at
FROM pitr_lab4
ORDER BY id;
"
```

Do not depend on a specific identity value for the post-PITR row; PostgreSQL sequences/identity values are not guaranteed to be gapless.

15. Acceptance Checklist

Acceptance Test	Result
DB1 source validated as PostgreSQL 18.6 primary/writable	PASS
WAL archive directory created with postgres ownership and mode 700	PASS
wal_level = replica	PASS
archive_mode = on	PASS
archive_command validated	PASS
pg_stat_archiver failed_count = 0	PASS
Physical pg_basebackup completed	PASS
Three valid test rows inserted	PASS
Named restore point created	PASS
Accidental DELETE removed 3 rows	PASS
Required WAL through segment 000C archived	PASS
Base backup and WAL transferred to DB2	PASS
Previous DB2 PGDATA preserved	PASS
Base backup restored into DB2 PGDATA	PASS
WAL archive restored on DB2	PASS
recovery.signal configured	PASS
restore_command configured	PASS
Named recovery target configured	PASS
Recovery stopped at requested restore point	PASS
New timeline ID 4 selected	PASS
Archive recovery completed	PASS
Three deleted rows recovered	PASS
DB2 promoted and writable	PASS

FINAL LAB RESULT: PASS

16. Post-Lab Cleanup and Production Notes

16.1 Optional lab cleanup

Perform cleanup only after the recovered database and SOP evidence have been retained.

```
# Optional: remove transferred compressed packages after validation
sudo rm -f /tmp/lab4_basebackup.tar.gz
sudo rm -f /tmp/lab4_wal_archive.tar.gz

# Review the preserved pre-LAB-4 DB2 cluster before deciding whether to remove it
sudo du -sh /var/lib/pgsql/data_before_lab4_pitr_20260902
```

16.2 Production considerations

- The local WAL archive used in this lab is intentionally simple. It is not disaster-safe because it resides on the same server as DB1.
- Production backup repositories should be durable, independently protected, monitored, capacity-managed, and regularly recovery-tested.
- A successful base backup alone is insufficient for PITR; every required WAL segment from the backup start through the recovery target must remain available.
- Monitor `pg_stat_archiver` and alert on failed archiving or an archive backlog.
- Use a dedicated backup product or repository design appropriate for the environment; PostgreSQL PITR procedures should be rehearsed regularly.
- Keep recovery targets deterministic. Named restore points are particularly useful for planned application/database changes.
- After recovery and promotion, PostgreSQL branches onto a new timeline. Preserve timeline history as part of the WAL archive strategy.
- Do not use `recovery.conf` with PostgreSQL 18. Recovery parameters are server configuration parameters and `recovery.signal` initiates archive recovery.

17. Official PostgreSQL References

- [PostgreSQL 18 - pg_basebackup](#)
- [PostgreSQL 18 - Backup and Restore](#)
- [PostgreSQL - Continuous Archiving and PITR](#)
- [PostgreSQL - WAL Configuration](#)
- [PostgreSQL 18 - Recovery Configuration](#)

Document Control

Field	Value
Document	LAB-4 PostgreSQL 18.6 PITR on AWS EC2
Status	Final - Tested
Result	PASS
Tested By	Praveen Madupu
Date	02 September 2026 - Wednesday
Scope Control	Only working/tested lab commands included